Concurrency Model	**Definition** Beschreibt die grundlegende Strategie eines Systems, wie mehrere unabhängig laufende Abläufe innerhalb eines Programms organisiert werden. Dazu gehört, ob Aufgaben parallel oder kooperativ ausgeführt werden, wie sie miteinander kommunizieren und wie Ressourcen gemeinsam genutzt werden können.	**Warum wichtig** Sobald Programme mehrere Aufgaben gleichzeitig ausführen sollen, entstehen Fragen nach Zuständigkeiten, Reihenfolgen und Zugriff auf gemeinsame Daten. Unterschiedliche Strategien führen zu unterschiedlichen Eigenschaften hinsichtlich Performance, Fehlerrisiken, Verständlichkeit und Wartbarkeit. Architekturentscheidungen hängen stark davon ab, welches Modell eingesetzt wird.
Process (OS)	**Definition** Eine eigenständige Ausführungseinheit eines Programms mit eigenem Adressraum und eigener Ressourcenverwaltung durch das Betriebssystem. Mehrere solcher Einheiten können gleichzeitig laufen, ohne direkt auf denselben Speicherbereich zuzugreifen.	**Warum wichtig** Die strikte Trennung der Speicherbereiche erhöht Stabilität und Sicherheit, weil Fehler in einer Ausführungseinheit andere nicht unmittelbar beeinflussen. Gleichzeitig entstehen zusätzliche Anforderungen an Kommunikation zwischen getrennten Bereichen. Diese Eigenschaften spielen eine zentrale Rolle bei Architekturentscheidungen verteilter oder isolierter Systeme.

Thread	**Definition** Eine kleinere Ausführungseinheit innerhalb desselben Adressraums eines Programms. Mehrere solche Einheiten können gleichzeitig aktiv sein und greifen dabei auf denselben Speicherbereich zu.	**Warum wichtig** Gemeinsam genutzter Speicher ermöglicht effiziente Zusammenarbeit zwischen gleichzeitig laufenden Abläufen. Gleichzeitig entsteht das Risiko von Inkonsistenzen durch konkurrierende Zugriffe auf Daten. Deshalb sind geeignete Koordinationsmechanismen notwendig, um korrektes Verhalten sicherzustellen.
Coroutine	**Definition** Eine Ausführungseinheit, die ihre Kontrolle freiwillig an andere Abläufe abgibt und später an derselben Stelle fortgesetzt werden kann. Der Wechsel zwischen mehreren Abläufen erfolgt dabei kooperativ statt durch parallele Betriebssystemsteuerung.	**Warum wichtig** Diese Technik ermöglicht effiziente Bearbeitung vieler gleichzeitig wartender Aufgaben, etwa bei Ein- und Ausgabeoperationen oder Netzwerkkommunikation. Dadurch lassen sich große Mengen paralleler Abläufe verwalten, ohne den Aufwand klassischer paralleler Ausführungseinheiten.

Lock	**Definition** Ein Mechanismus, der sicherstellt, dass ein bestimmter Programmabschnitt nur von einer Ausführungseinheit gleichzeitig betreten werden kann. Andere Einheiten müssen warten, bis der Zugriff wieder freigegeben wird.	**Warum wichtig** Wenn mehrere gleichzeitig aktive Abläufe auf dieselben Daten zugreifen, können widersprüchliche Zustände entstehen. Eine solche Zugriffsbeschränkung verhindert unkontrollierte Überschreibungen und stellt sicher, dass Änderungen vollständig und konsistent erfolgen.
Semaphore	**Definition** Ein Zählmechanismus zur Steuerung des Zugriffs mehrerer gleichzeitig aktiver Abläufe auf begrenzte Ressourcen. Dabei wird festgelegt, wie viele Zugriffe gleichzeitig erlaubt sind.	**Warum wichtig** Viele Ressourcen stehen nur in begrenzter Anzahl zur Verfügung, etwa Verbindungen oder Gerätezugriffe. Durch kontrollierte Vergabe von Zugriffsrechten lassen sich Überlastung, Konflikte und inkonsistente Zustände vermeiden. Dadurch bleibt das Verhalten eines Systems auch bei hoher Belastung kontrollierbar.

Serialization	**Definition** Ein Verfahren zur Umwandlung komplexer Datenstrukturen in eine übertragbare oder speicherbare Darstellung, etwa in Textform oder Binärform, sodass sie außerhalb des aktuellen Programmlaufs gespeichert oder übertragen werden können.	**Warum wichtig** Daten müssen häufig zwischen Programmen, über Netzwerke oder in Dateien ausgetauscht werden. Ohne eine geeignete Umwandlung wären komplexe interne Strukturen nicht transportierbar. Dieses Verfahren bildet daher eine Grundlage für Persistenz, Kommunikation zwischen Systemen und langfristige Speicherung von Zuständen.
ORM	**Definition** Eine Technik zur automatischen Zuordnung zwischen objektorientierten Datenstrukturen eines Programms und tabellenbasierten Datenstrukturen relationaler Datenbanksysteme. Dabei werden Objekte in Tabellenzeilen übersetzt und umgekehrt.	**Warum wichtig** Programme arbeiten häufig mit Objekten, während Datenbanksysteme Tabellen verwenden. Ohne eine solche Zuordnung müsste jede Speicherung und Abfrage manuell umgesetzt werden. Eine automatisierte Abbildung vereinfacht Zugriff, reduziert Fehlerquellen und sorgt für klarere Trennung zwischen Programmlogik und Datenhaltung.

Import	**Definition** Eine Sprachkonstruktion, mit der Bestandteile aus anderen Dateien, Modulen oder Paketen im eigenen Quelltext verfügbar gemacht werden. Dadurch lassen sich dort definierte Funktionen, Klassen oder Konstanten verwenden, ohne sie erneut implementieren zu müssen.	**Warum wichtig** Moderne Software besteht aus vielen getrennten Bausteinen, die miteinander kombiniert werden. Durch gezieltes Einbinden vorhandener Bestandteile bleibt Code übersichtlich strukturiert, Wiederverwendung wird möglich und große Projekte lassen sich in klar abgegrenzte Teile zerlegen.
Library	**Definition** Eine Sammlung vorgefertigter Funktionen, Klassen oder Module, die unabhängig vom eigenen Projekt entwickelt wurden und typische Aufgaben wie Datenverarbeitung, Benutzeroberflächen oder Kommunikation bereitstellen.	**Warum wichtig** Viele Anforderungen lassen sich effizienter lösen, wenn bereits vorhandene und erprobte Bausteine genutzt werden. Dadurch sinkt der Implementierungsaufwand, bewährte Lösungen können übernommen werden und komplexe Funktionalität steht schneller zur Verfügung.

Exception	**Definition** Ein besonderes Ereignis während der Programmausführung, das anzeigt, dass eine erwartete Operation nicht wie vorgesehen abgeschlossen werden konnte. Der normale Ablauf wird dabei unterbrochen und an eine vorgesehene Behandlungsstruktur weitergeleitet.	**Warum wichtig** Viele Fehler entstehen erst während der Ausführung eines Programms. Ein strukturierter Umgang mit solchen Situationen ermöglicht kontrollierte Reaktionen statt unvorhersehbarer Programmabbrüche und verbessert Stabilität sowie Nachvollziehbarkeit des Verhaltens.
Interpreter	**Definition** Ein Programm, das Anweisungen schrittweise verarbeitet und direkt während der Ausführung umsetzt, ohne zuvor ein vollständig eigenständiges ausführbares Programm zu erzeugen.	**Warum wichtig** Dieses Vorgehen ermöglicht unmittelbare Rückmeldungen während der Entwicklung und erleichtert experimentelles Arbeiten. Gleichzeitig ergeben sich Unterschiede bei Geschwindigkeit, Fehlersuche und Verteilung von Programmen im Vergleich zu anderen Übersetzungsstrategien.

Compiler	**Definition** Ein Programm, das Quelltext vollständig in eine ausführbare Zielrepräsentation überführt, bevor die eigentliche Programmausführung beginnt.	**Warum wichtig** Die vollständige Übersetzung vor dem Start ermöglicht frühzeitige Fehlererkennung und führt häufig zu effizienterem Laufzeitverhalten. Dadurch ergeben sich andere Anforderungen an Entwicklungsabläufe und Bereitstellung von Software.
Speicherverwaltung	**Definition** Die Organisation der Nutzung des Arbeitsspeichers während der Programmausführung. Dabei wird festgelegt, wann Daten angelegt, verändert und wieder freigegeben werden.	**Warum wichtig** Programme arbeiten ständig mit Daten im Arbeitsspeicher. Eine strukturierte Nutzung verhindert unnötigen Ressourcenverbrauch, reduziert Fehlerquellen und beeinflusst maßgeblich Stabilität sowie Leistungsfähigkeit eines Systems.

Garbage Collection	**Definition** Ein Verfahren, bei dem eine Laufzeitumgebung selbstständig erkennt, welche Datenbereiche nicht mehr verwendet werden, und deren Speicher automatisch wieder freigibt.	**Warum wichtig** Automatische Freigabe reduziert Fehler, die durch vergessene Speicherfreigaben entstehen können. Gleichzeitig beeinflusst sie das Laufzeitverhalten eines Programms und verändert den Umgang mit Ressourcen gegenüber stärker manuell kontrollierten Umgebungen.
IDE	**Definition** Eine Software, die zentrale Tätigkeiten der Programmierung wie Schreiben, Navigieren, Ausführen und Analysieren von Quelltext in einer gemeinsamen Arbeitsoberfläche zusammenführt.	**Warum wichtig** Komplexe Projekte bestehen aus vielen Dateien und Abhängigkeiten. Eine gemeinsame Arbeitsumgebung erleichtert Orientierung im Projekt, beschleunigt typische Arbeitsabläufe und unterstützt strukturierte Entwicklung.

Versionskontrolle	**Definition** Ein System zur strukturierten Nachverfolgung von Änderungen an Dateien über die Zeit hinweg, einschließlich der Möglichkeit, frühere Zustände wiederherzustellen und parallele Änderungen zusammenzuführen.	**Warum wichtig** Änderungen müssen nachvollziehbar bleiben, insbesondere wenn mehrere Arbeitsstände oder mehrere Beteiligte beteiligt sind. Eine strukturierte Nachverfolgung ermöglicht sichere Weiterentwicklung und erleichtert die Analyse von Fehlerursachen.
Package dependencies	**Definition** Externe Softwarebestandteile, deren Vorhandensein und passende Version erforderlich sind, damit ein Projekt korrekt ausgeführt werden kann.	**Warum wichtig** Viele Programme nutzen zahlreiche externe Bausteine. Nur wenn deren Versionen eindeutig festgelegt und verwaltet werden, bleibt ein Projekt zuverlässig reproduzierbar und stabil betreibbar.

Debugger	**Definition** Ein Werkzeug, mit dem Programme kontrolliert ausgeführt werden können, sodass interne Zustände sichtbar werden und einzelne Schritte der Ausführung gezielt untersucht werden können.	**Warum wichtig** Viele Fehler lassen sich nur erkennen, wenn der tatsächliche Ablauf während der Ausführung nachvollzogen wird. Sichtbarkeit interner Zustände ermöglicht systematische Analyse komplexer Probleme.
SOLID principles	**Definition** Eine Sammlung von fünf Entwurfsprinzipien für objektorientierte Systeme, die auf klare Zuständigkeiten, flexible Erweiterbarkeit und reduzierte Abhängigkeiten zwischen Programmteilen abzielen. (Single Responsibility, Open Closed, Liskov Substitution, Interface Segregation, Dependency Inversion)	**Warum wichtig** Strukturelle Entscheidungen wirken sich langfristig auf Wartbarkeit und Erweiterbarkeit aus. Klare Aufgabentrennung und kontrollierte Abhängigkeiten erleichtern Anpassungen und reduzieren unerwartete Seiteneffekte.

DRY principle	**Definition** Eine Leitidee, nach der fachliche Regeln und Logik möglichst nur an einer zentralen Stelle im Code definiert werden sollen statt mehrfach unabhängig voneinander.	**Warum wichtig** Mehrfach vorhandene Logik erhöht den Wartungsaufwand und führt leicht zu widersprüchlichem Verhalten bei Änderungen. Eine zentrale Definition erleichtert Anpassungen und verbessert Konsistenz.
Consistent level of abstraction	**Definition** Beschreibt die Strukturierung von Programmteilen so, dass innerhalb eines Abschnitts Aussagen auf vergleichbarer gedanklicher Detailstufe formuliert sind und unterschiedliche Ebenen nicht unkontrolliert vermischt werden.	**Warum wichtig** Klare Ebenentrennung verbessert Lesbarkeit und erleichtert Verständnis komplexer Abläufe. Dadurch lassen sich Programme besser strukturieren und Änderungen gezielter durchführen.

Interfaces	**Definition** Klar definierte Übergänge zwischen Programmteilen, bei denen festgelegt ist, welche Eingaben erwartet werden und welche Ergebnisse bereitgestellt werden.	**Warum wichtig** Klare Übergänge zwischen Komponenten ermöglichen unabhängige Entwicklung einzelner Teile und reduzieren enge Verflechtungen innerhalb eines Systems. Dadurch bleiben Programme besser wartbar und erweiterbar.